What's new

A Lean companion to "Analysis I"

Terence Tao · 31 May 2025

https://terrytao.wordpress.com/2025/05/31/a-lean-companion-to-analysis-i/

tao2typ · converted to Typst 2026-10-14 (UTC)

Almost 20 years ago, I wrote a textbook in real analysis called "Analysis I". It was intended to complement the many good available analysis textbooks out there by focusing more on foundational issues, such as the construction of the natural numbers, integers, rational numbers, and reals, as well as providing enough set theory and logic to allow students to develop proofs at high levels of rigor.

While some proof assistants such as Coq or Agda were well established when the book was written, formal verification was not on my radar at the time. However, now that I have had some experience with this subject, I realize that the content of this book is in fact very compatible with such proof assistants; in particular, the 'naive type theory' that I was implicitly using to do things like construct the standard number systems, dovetails well with the dependent type theory of Lean (which, among other things, has excellent support for quotient types).

I have therefore decided to launch a Lean companion to "Analysis I", which is a "translation" of many of the definitions, theorems, and exercises of the text into Lean. In particular, this gives an alternate way to perform the exercises in the book, by instead filling in the corresponding "sorries" in the Lean code. (I do not however plan on hosting "official" solutions to the exercises in this companion; instead, feel free to create forks of the repository in which these sorries are filled in.)

Currently, the following sections of the text have been translated into Lean:

- Section 2.1: The natural numbers
- Section 2.2: Addition
- Section 2.3: Multiplication
- Chapter 2 epilogue: Isomorphism with the Mathlib natural numbers
- Section 3.1: Basic set theory
- Section 4.1: The integers

The formalization has been deliberately designed to be separate from the standard Lean math library Mathlib at some places, but reliant on it at others. For instance, Mathlib already has a standard notion of the natural numbers $\mathbf{N}$. In the Lean formalization, I first develop "by hand" an alternate construction `Chapter2.Nat` of the natural numbers (or just `Nat`, if one is working in the `Chapter2` namespace), setting up many of the basic results about these alternate natural numbers which parallel similar lemmas about $\mathbf{N}$ that are already in Mathlib (but with many of these lemmas set as exercises to the reader, with the proofs currently replaced with "sorries"). Then, in an epilogue section, isomorphisms between these alternate natural numbers and the Mathlib natural numbers are established (or more precisely, set as exercises). From that point on, the Chapter 2 natural numbers are deprecated, and the Mathlib natural numbers are used instead. I intend to continue this general pattern throughout the book, so that as one advances into later chapters, one increasingly relies on Mathlib's definitions and functions, rather than directly referring to any counterparts from earlier chapters. As such, this companion could also be used as an introduction to Lean and Mathlib as well as to real analysis (somewhat in the spirit of the "Natural number game", which in fact has significant thematic overlap with Chapter 2 of my text).

The code in this repository compiles in Lean, but I have not tested whether all of the (numerous) "sorries" in the code can actually be filled (i.e., if all the exercises can actually be solved in Lean). I would be interested in having volunteers "playtest" the companion to see if this can actually be done (and if the helper lemmas or "API" provided in the Lean files are sufficient to fill in the sorries in a conceptually straightforward manner without having to rely on more esoteric Lean programming techniques). Any other feedback will of course also be welcome.

[UPDATE, May 31: moved the companion to a standalone repository.]

Comments (31)

Anonymous · 31 May 2025

Your book sounds similar to Edmund Landau's Grundlagen der Analysis in its objectives. He constructs in succession the number systems from the naturals to the complex numbers using tools such as quotienting and Dedekind cuts. This book was formalised by Jutting in 1977 to demonstrate the power of de Bruijn's AUTOMATH system.

> **Hendrik Boom** · 31 May 2025
>
> Jutting did formalize the entire Grundlagen into the Automath version of type theory. Many of the documents relating to this have been collected at https://automath.win.tue.nl/

Guanyuming He · 31 May 2025

Dear Prof. Tao,
Would you like to accept contributions of new Exercises, or more generally, interesting propositions, to the companion?

> **Terence Tao** · 31 May 2025
>
> I think these would be better suited for downstream forks of the repository (and similarly for solutions to exercises); I plan to keep the core repository faithful to the original source material.
>
> > **Anonymous** · 4 June 2025
> >
> > Maybe there can be something like a community appendix with additional exercises.
> >
> > > **Terence Tao** · 4 June 2025
> > >
> > > I have no objection to a community fork of the repository to hold this sort of additional content (and I believe the Apache 2.0 licence I selected is compatible with this).

Anonymous · 31 May 2025

The links to the different sections doesn't seem to work for me. I get 404.
[Fixed, thanks -T.]

> **Anonymous** · 31 May 2025
>
> The link behind "Web page for this project" should be "https://teorth.github.io/analysis/", not "https://teorth.github.io/Analysis/".
>
> > **Anonymous** · 4 June 2025
> >
> > The broken link I was talking about is in the following file: "https://github.com/teorth/analysis/blob/main/README.md"
> > *[This has now been fixed. In general, fixes to the Github repository are best handled through the issues and pull request pages. -T]*

Peter Gent · 31 May 2025

Thanks for sharing this project, Prof. Tao. *Analysis I* is my favourite maths textbook. It helped me learn a lot about mathematics, but more importantly, it taught me about clear writing and exposition. There's something poetic about seeing some of Leibniz's most elegant results now being formalized in a way that AI systems can readily interact with them. It's his *calculus ratiocinator* coming to life.

Anonymous · 31 May 2025

Is Type theory of VV related?

Vatsa · 1 June 2025

This should be a course at UCLA!
Here's an analogous course on programming language theory
https://plfa.github.io/

Anonymous · 1 June 2025

related (in spirit):
"The Number Systems: Foundations of Algebra and Analysis" by Solomon Feferman: https://bookstore.ams.org/chel-333/

thmataignef165189892 · 2 June 2025

I am willing to use Lean to follow your work, but after installing it on Windows 11 my anti-virus G-Data complains about several files:
Virus: Win64.Trojan.Agent.XXMXQQ (Moteur B)Fichier: lean.exeDossier: C:Usersthmat.elanbin
No real problem for me, I just asked to put the files in quarantine, but I am a bit surprised.

Anonymous · 2 June 2025

I think the section 2.2 is still pointing to the older location. It should be https://teorth.github.io/analysis/docs/Analysis/Section_2_2.html
-Nagu
[Corrected, thanks – T.]

Daily Links: Tuesday, Jun 3rd, 2025 | From Pixels to Particles · 3 June 2025

[…] A Lean companion to "Analysis I": Almost 20 years ago, I wrote a textbook in real analysis called "Analysis I". It was intended to complement the many good available analysis textbooks out there by focusing more on foun… […]

Double Maths First Thing: Issue 27 | The Aperiodical · 3 June 2025

[…] the Fields medallists doing interesting things beat, Terry Tao has launched a Lean companion to Analysis I — Lean is a "formal verification system" that checks your proofs hold water. […]

Anonymous · 16 June 2025

Hello this seems quite interesting and I've started working through it.
What sorts of comments on the exercises do you find useful?
I found that 2.2.10 was quite hard from a lean perspective as you just got dropped in the deep end of proving an exists unique with no other examples surrounding it proving this new syntax and also I thought 2.2.12 (b) (greater than is transitive) was a bit hard as I think the way you need to do it is to get the value out of the ∃ symbol in each of the two hypotheses but there weren't any examples of doing that in the surrounding text. (I used obtain to do this)

> **Terence Tao** · 17 June 2025
>
> Thanks, this is useful feedback! I have been focusing on formalizing the statements of the exercises and the main text, and have not had the time to "playtest" the proofs of the exercises, so this sort of field report is very useful. I've added some comments to Section 2.2 along these lines. In the future I may end up formalizing Appendix A (the basics of mathematical logic), which could also serve as a dual role of providing some introduction to Lean basics such as the `obtain` tactic. But for now my main focus is on formalizing the "analysis" part of the text (I just finished

sequences, am currently starting on series, and then there is also continuity, differentiation, and integration to get to).

 **Anonymous** · 17 June 2025

Dear Professor Tao, Thank you for your efforts in formalizing proofs. I am personally not familiar with Lean, but I have a general question. Say Group 1 proves a Theorem A and it's peer-reviewed and published in a reputable journal. Then say a Group 2 formalizes it, and they conclude that it's incorrect. How can one verify whether Group 2 formalizes is correct or whether they made a mistake in coding it. I am just personally concerned about the false positive situation in Lean. Can someone "prove" that their formalizes is correct? Thank you!

 **Terence Tao** · 18 June 2025

If both groups formalized in the same version of Lean, one can try to merge the two proofs together and see if one can obtain a genuine contradiction. Most likely, what would have happened is that one (or both) of the groups would have formalized a definition, or the statement of the result, incorrectly, for instance by not handling an edge case (e.g., if a key parameter vanishes) correctly. Probably the effort of trying to reconcile the results (by trying to find a "minimal working example" (MWE) of the contradiction) will zero in on the precise discrepancy, in a way that would be faster than if one or both proofs were informal.

If the two proofs were formalized in incompatible formal languages (e.g., Lean and Rocq) then the situation is more complicated, but one could still try to isolate subtheorems that can be proven true in one formalization and false in another, again to try to find the MWE of the contradiction. It may be a time-consuming process, but having the ability to formally verify consequences of either formalization will make this a more efficient (and hopefully less acrimonious) process than if one only had informal proofs to work with.

 **JL** · 19 June 2025

I think what Anonymous is asking about is the situation where one of the group writes a pen and paper proof and the other writes the proof in Lean. They are basically asking if it's possible to prove that the Lean compiler is correct.

To answer that question: kind of. You can actually prove mathematically that a piece of code does what you want it to do. So you could prove the code of the Lean compiler is correct. But then what about the compiler that compiles the Lean compiler? You can prove that to be correct too. But then there is the compiler that compiles the compiler that compiles the Lean compiler. It is basically turtles all the way down (though I guess in theory you could keep going down the chain of compilers until you go so low that you start proving things about circuits, but this is practically impossible).

In practice, since Lean is widely used, if there was some sort of bug, it would probably be brought to attention and fixed. Of course, this isn't a 100% guarantee, but I personally would definitely trust the Lean proof over the pen and paper proof.

 **Guanyuming He** · 22 June 2025

Dear JL,

> You can actually prove mathematically that a piece of code does what you want it to do.

Can we? I have the impression that there's no way to formally prove that a formal system is a faithful abstraction of our informal understanding of something, as we can't formally reason about things in our brain. After all, we have to have an idea of meaning when we

want to prove a formal system means what we mean, and for that we can only rely on our informal and intuitive understanding of it inside our brain.

 **Terence Tao** · 15 July 2025

Analysis I is now completely formalized (except for the introduction chapter, which has a significantly less formal style and is more concerned with (incorrect) informal arguments that do *not* translate into formal proofs).

> A Lean companion to "Analysis I"

 **Anonymous** · 17 July 2025

Happy Birthday to Terence Tao :)

 **Anonymous** · 2 August 2025

If it's ok to ask this here, why has the math world moved from Rocq to Lean? Are they much different? I've looked at Rocq a little (the Software Foundations books) and I know it's been used for some major formalizations.

 **Terence Tao** · 8 August 2025

I came to this subject only the last few years, so cannot speak with much authority on the early history of this, but my sense is that Rocq is more well adapted to computer science applications, such as formally verifying the soundness of computer languages, while Lean (particularly with its Mathlib model of a single, standardized library) is more well adapted to how a working mathematician actually writes and thinks about proofs. Certainly I found the design choices of Lean and Mathlib to harmonize quite well with my undergraduate Analysis I textbook when I formalized it a few months ago: for instance, Lean's support of quotient types made the construction of number systems such as the integers, rationals, or reals quite straightforward, with a pretty close match between what was in my text (which was in some sense implementing a "naive" version of type theory with quotient types) with the formalism in Lean. I have not attempted to accomplish a similar exercise in Rocq, but my understanding is that they made some different fundamental design choices, in which quotients were made more difficult to work with in order to preserve some more technical type theoretic properties of the language of interest to computer scientists.

That said, I hope that advances in auto-formalization will eventually allow proofs in one formal language such as Rocq to be translated semi-automatically to another language such as Lean (or vice versa), though there are some fundamental issues here regarding differences in idiomatic proof writing style, and substantially different libraries for basic mathematical concepts such as the real numbers, which are non-trivial issues to overcome here.

 **Anonymous** · 20 August 2025

this seems like a big deal:
https://x.com/sebastienbubeck/status/1958198661139009862?s=46

 **Anonymous** · 28 September 2025

Would formalizing Analysis II be more difficult than Analysis I? How about the graduate analysis sequence 245ABC?

 **Terence Tao** · 29 September 2025

I am teaching 245A (measure theory) right now, and concurrently beginning the process of formalizing the text, see https://github.com/teorth/analysis/tree/main/analysis/Analysis/

MeasureTheory . I am hoping to get some of the students in the class involved in the process, and maybe also test out some newer autoformalization tools. If successful, I could try to replicate this for Analysis II, but this is not currently a priority for me.



Anonymous · 27 March 2026

This is great! Are you interested in pull requests for the ‘sorry’s in the proofs (that weren’t left as exercises to the reader). I haven’t gotten too far in, but I see a couple in WellFoundedLT.partialOrder (Lemma 8.5.14).
Thanks.